

Optimization algorithms

Mini-batch Gradient Descent

Vectorization allows you to process all M examples relatively quickly if M is very large then it can still be slow.

Batch vs. mini-batch gradient descent

Vectorization allows you to efficiently compute on m examples.

$$\begin{aligned}
 X &= \begin{bmatrix} x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(1000)} & \dots & x^{(1001)} & \dots & x^{(2000)} & \dots & \dots & x^{(m)} \end{bmatrix} \\
 &\quad (n_x, m) \quad \underbrace{\hspace{10em}}_{X^{t13}} \quad \underbrace{\hspace{10em}}_{X^{t23}} \quad \dots \quad \underbrace{\hspace{10em}}_{X^{t5,0003}} \\
 Y &= \begin{bmatrix} y^{(1)} & y^{(2)} & y^{(3)} & \dots & y^{(1000)} & \dots & y^{(1001)} & \dots & y^{(2000)} & \dots & \dots & y^{(m)} \end{bmatrix} \\
 &\quad (1, m) \quad \underbrace{\hspace{10em}}_{Y^{t13}} \quad \underbrace{\hspace{10em}}_{Y^{t23}} \quad \dots \quad \underbrace{\hspace{10em}}_{Y^{t5,0003}}
 \end{aligned}$$

What if $m = 5,000,000$?
 5,000 mini-batches of 1,000 each
 Mini-batch t : X^{t13}, Y^{t13}

$$\begin{aligned}
 &X^{(i)} \\
 &z^{[l]} \\
 &X^{t13}, Y^{t13}
 \end{aligned}$$

Andrew Ng

For example what if M was 5 million or 50 million or even bigger.

With the implementation of gradient descent on your whole training set, what you have to do is, you have to process your entire training set before you take one little step of gradient descent.

And then you have to process your entire training sets of five million training samples again before you take another little step of gradient descent.

So, it turns out that you can get a faster algorithm if you let gradient descent start to make some progress even before you finish processing your entire, your giant training sets of 5 million examples. In particular, here's what you can do. Let's say that you split up your training set into smaller, little baby training sets and these baby training sets are called mini-batches.

And let's say each of your baby training sets have just 1,000 examples each.

So, you take X_1 through $X_{1,000}$ and you call that your first little baby training set, also call the mini-batch. And then you take home the next 1,000 examples. $X_{1,001}$ through $X_{2,000}$ and the next $X_{1,000}$ examples and come next one and so on.

I'm going to introduce a new notation. I'm going to call this X superscript with curly braces $\{\}$, 1 and I am going to call this, X superscript with curly braces, 2. Now, if you have 5 million training samples total and each of these little mini batches has a thousand examples, that means you have 5,000 of these because you know, 5,000 times 1,000 equals 5 million.

Altogether you would have 5,000 of these mini batches.

Batch vs. mini-batch gradient descent

Vectorization allows you to efficiently compute on m examples.

$$\begin{aligned}
 X &= \begin{bmatrix} x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(1000)} & \dots & x^{(1001)} & \dots & x^{(2000)} & \dots & \dots & x^{(m)} \end{bmatrix} \\
 &\quad (n_x, m) \quad \underbrace{\hspace{10em}}_{X^{t13} (n_x, 1000)} \quad \underbrace{\hspace{10em}}_{X^{t23} (n_x, 1000)} \quad \dots \quad \underbrace{\hspace{10em}}_{X^{t5,0003} (n_x, 1000)} \\
 Y &= \begin{bmatrix} y^{(1)} & y^{(2)} & y^{(3)} & \dots & y^{(1000)} & \dots & y^{(1001)} & \dots & y^{(2000)} & \dots & \dots & y^{(m)} \end{bmatrix} \\
 &\quad (1, m) \quad \underbrace{\hspace{10em}}_{Y^{t13} (1, 1000)} \quad \underbrace{\hspace{10em}}_{Y^{t23} (1, 1000)} \quad \dots \quad \underbrace{\hspace{10em}}_{Y^{t5,0003} (1, 1000)}
 \end{aligned}$$

What if $m = 5,000,000$?
 5,000 mini-batches of 1,000 each
 Mini-batch t : X^{t13}, Y^{t13}

$$\begin{aligned}
 &X^{(i)} \\
 &z^{[l]} \\
 &X^{t13}, Y^{t13}
 \end{aligned}$$

Andrew Ng

$x^{(i)}$: The training sample at i -th

$z^{[l]}$: The layer l

$X^{\{t\}}$: the mini-batch t

if the shape of X is (n, m) where n is number of features in X , m is number of training samples, the $X^{\{1\}}$ has the shape $(n, 1000)$ cuz we choose batch size is 1000 training samples

Mini-batch gradient descent

for $t = 1, \dots, 5000$ {

Forward prop on $X^{\{t\}}$.

$$Z^{\{t\}} = W^{\{t-1\}} X^{\{t\}} + b^{\{t-1\}}$$

$$A^{\{t\}} = \sigma(Z^{\{t\}})$$

Vertical implmentation (1000 examples)

$$A^{\{t\}} = \sigma(Z^{\{t\}})$$

Compute cost $J^{\{t\}} = \frac{1}{1000} \sum_{i=1}^{1000} \ell(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_{i=1}^{1000} \|W^{(i)}\|_F^2$

Backprop to compute gradients w.r.t $J^{\{t\}}$ (using $(X^{\{t\}}, Y^{\{t\}})$)

$$W^{\{t\}} = W^{\{t-1\}} - \alpha dW^{\{t\}}, \quad b^{\{t\}} = b^{\{t-1\}} - \alpha db^{\{t\}}$$

}

1 step of gradient descent using $X^{\{t\}}, Y^{\{t\}}$ (as if $m=1000$)

X, Y

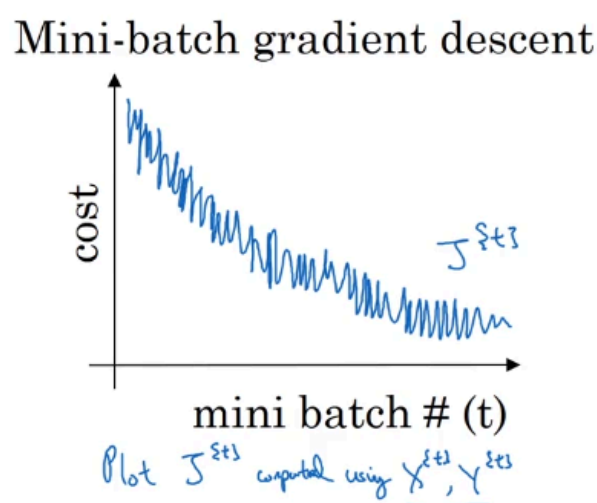
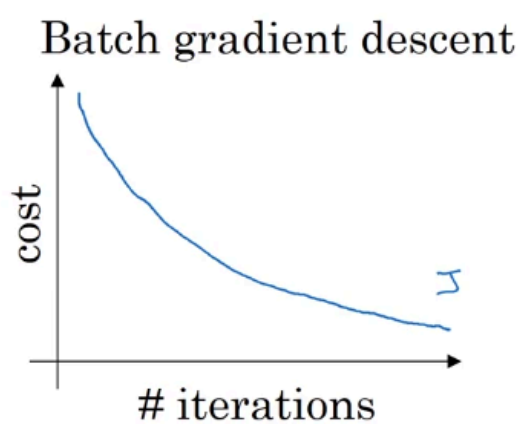
Andrew Ng

This is one pass through your training set using mini-batch gradient descent. The code I have written down here is also called doing one epoch of training and epoch is a word that means a single pass through the training set.

Whereas with batch gradient descent, a single pass through the training set allows you to take only one gradient descent step. With mini-batch gradient descent, a single pass through the training set, that is one epoch, allows you to take 5,000 gradient descent steps. Now of course you want to take multiple passes through the training set which you usually want to, you might want another for loop for another while loop out there.

So you keep taking passes through the training set until hopefully you converge or at least approximately converged.

Understanding Mini-batch Gradient Descent

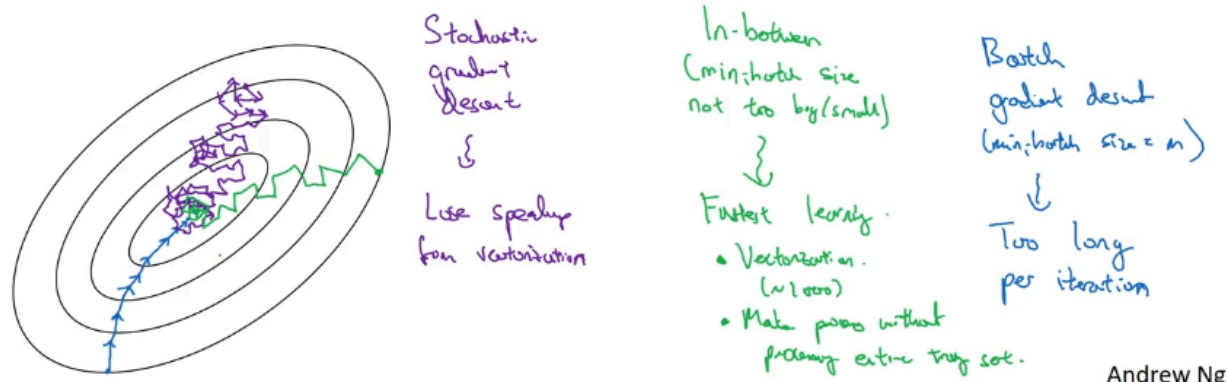


Andrew Ng

Choosing your mini-batch size

Choosing your mini-batch size

- If mini-batch size = m : Batch gradient descent. $(X^{(1)}, Y^{(1)}) = (X, Y)$
- If mini-batch size = 1 : Stochastic gradient descent. Every example is its own mini-batch.
 $(X^{(1)}, Y^{(1)}) = (x^{(1)}, y^{(1)}) \dots (x^{(n)}, y^{(n)})$ mini-batch.
- In practice: Somewhere in-between 1 and m



- If mini-batch size = m : Batch gradient descent
- If mini-batch size = 1: Stochastic gradient descent.

In practice:

- if we use stochastic gradient descent: We can lose the speed up from vectorization
- If we use Batch gradient descent (mini-batch with size = m , where m is the number of training samples): it's gonna take too long per iteration
- If we use in between 1 and m (mini-batch size not too small/big): This is the fastest training
 - We get a lot of vectorization
 - You can also make progress, Without needing to wait til you process the entire training set. So again using the numbers we have from the previous video, each epoch each part your training set allows you to see 5,000 gradient descent steps

How to choose a good mini-batch size?

- If we have a small training set: Use batch gradient descent ($m \leq 2000$)
- Big training set: typical mini-batch size (64, 128, 256, 512, 1024 → this kinda rare)
- Make sure mini-batch fit in CPU/GPU Memory

In practice, of course the mini-batch size is another hyper parameter that you might do a quick search over to try to figure out which one is most sufficient of reducing the cost function J

Exponentially Weighted Averages

- Few optimization algorithms → faster than gradient descent.

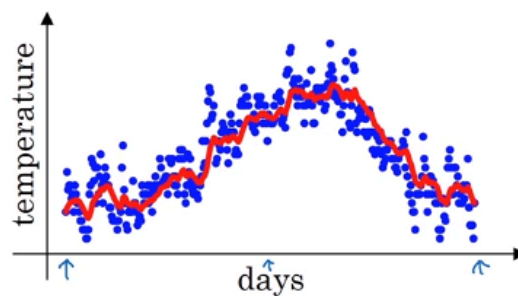
But in order to understand those algorithms, we need to be able to use 'Weighted averages'

Or also called exponentially weighted moving averages in statistics.

Temperature in London

Temperature in London

$\theta_1 = 40^\circ\text{F}$ $4^\circ\text{C} \leftarrow$
 $\theta_2 = 49^\circ\text{F}$ 9°C
 $\theta_3 = 45^\circ\text{F}$ $\vdots$
 $\vdots$
 $\theta_{180} = 60^\circ\text{F}$ 15°C
 $\theta_{181} = 56^\circ\text{F}$ $\vdots$
 $\vdots$



$$\begin{aligned}
 v_0 &= 0 \\
 v_1 &= 0.9 v_0 + 0.1 \theta_1 \\
 v_2 &= 0.9 v_1 + 0.1 \theta_2 \\
 v_3 &= 0.9 v_2 + 0.1 \theta_3 \\
 &\vdots \\
 v_t &= 0.9 v_{t-1} + 0.1 \theta_t
 \end{aligned}$$

Andrew Ng

This data looks a little bit noisy! We want to compute the trends, the local average or a moving average of the temperature.

Let's initialize $V = 0$. And then, on every day, we're going to average it with a weight of 0.9 times whatever appears as value, + 0.1 times that day temperature. So, theta one here would be the temperature from the first day.

continue like this:

we have $V_t = 0.9 * V_{t-1} + 0.1 * \theta$

where, theta is the temperature of that day (the day t)

Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$

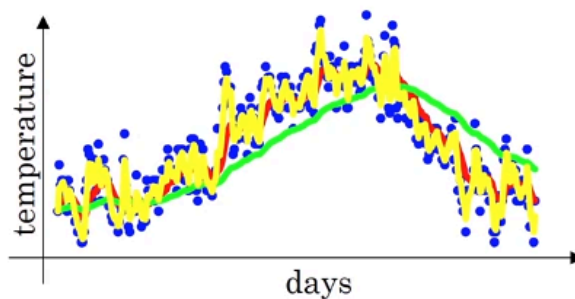
v_t : can approximate average over $\frac{1}{1-\beta}$ days temperature.

Exponentially weighted averages

$$\begin{aligned}
 v_t &= \beta v_{t-1} + (1-\beta) \theta_t \\
 \beta &= 0.9 & : \approx 10 \text{ days' temperature} \\
 \beta &= 0.98 & : \approx 50 \text{ days} \\
 \beta &= 0.5 & : \approx 2 \text{ days}
 \end{aligned}$$

v_t is an approximately
 average over
 $\rightarrow \approx \frac{1}{1-\beta}$ days'
 temperature.

$$\frac{1}{1-0.98} = 50$$



Andrew Ng

- The red line is for beta = 0.9
- The yellow line is for beta = 0.5
- and the green line is for beta = 0.98

Understanding Exponentially Weighted Averages

Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$

$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$

$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$

$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$

$$\dots$$

$$v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9(\dots)))$$

$$= 0.1\theta_{100} + 0.1 \times 0.9 \times \theta_{99} + 0.1(0.9)^2\theta_{98} + 0.1(0.9)^3\theta_{97} + \dots$$

$$0.9^{10} \approx 0.35 \approx \frac{1}{e}$$

$$\frac{(1-\epsilon)^{1/\epsilon}}{0.9} \approx \frac{1}{e}$$

$$\epsilon = 0.02 \rightarrow 0.98^{50} \approx \frac{1}{e}$$

Andrew Ng

Implementing exponentially weighted averages

$$v_0 = 0$$

$$v_1 = \beta v_0 + (1 - \beta) \theta_1$$

$$v_2 = \beta v_1 + (1 - \beta) \theta_2$$

$$v_3 = \beta v_2 + (1 - \beta) \theta_3$$

$$\dots$$

$$v_0 := 0$$

$$v_0 := \beta v + (1 - \beta) \theta_1$$

$$v_0 := \beta v + (1 - \beta) \theta_2$$

$$\vdots$$

$$\rightarrow v_0 = 0$$

$$\text{Repeat } \{$$

$$\text{Get next } \theta_t$$

$$v_0 := \beta v_0 + (1 - \beta) \theta_t \leftarrow$$

$$\}$$

Andrew Ng

1. What is an Average (for context)?

A *simple average* of values $x_1, x_2, \dots, x_t$ is:

$$v_t = \frac{1}{t} \sum_{i=1}^t x_i$$

All points are **equally weighted** — past and recent values count the same.

2. Why “Exponentially Weighted”?

Sometimes, **recent values** should matter more than older ones.

That’s what EWA does — it gives **exponentially decreasing weights** to older data points.

3. Formula

$$v_t = \beta v_{t-1} + (1 - \beta) x_t$$

Where:

- v_t : the exponentially weighted average at time t
- x_t : the new observation (current value)
- β : the **decay rate**, $0 < \beta < 1$
- v_{t-1} : the previous EWA value



🔍 4. How it works (Intuition)

Each new value x_t contributes a little to the new average.

Older values are multiplied by β , so their influence **fades exponentially** over time.

Let's unroll it:

$$v_t = (1 - \beta)x_t + \beta(1 - \beta)x_{t-1} + \beta^2(1 - \beta)x_{t-2} + \dots$$

You can see that:

- The newest value x_t gets weight $1 - \beta$
- The one before gets $\beta(1 - \beta)$
- The one before that gets $\beta^2(1 - \beta)$
- ... and so on.

So the weights decrease exponentially.

📊 5. Example

Let's say:

- $\beta = 0.9$
- Values: [10, 13, 15, 14, 16]

We start with $v_0 = 0$ (for simplicity).

t	x_t	v_t = 0.9v_{t-1} + 0.1x_t
1	10	0.9×0 + 0.1×10 = 1.0
2	13	0.9×1 + 0.1×13 = 2.2
3	15	0.9×2.2 + 0.1×15 = 3.48
4	14	0.9×3.48 + 0.1×14 = 4.59
5	16	0.9×4.59 + 0.1×16 = 5.73

The average *smoothly follows* the data but doesn't jump as sharply — it's "smoothed."

🔧 6. Bias Correction (in practice)

When $v_0 = 0$, early values are **biased toward 0**.

We fix it using:

$$v_t^{\text{corrected}} = \frac{v_t}{1 - \beta^t}$$

This is used in algorithms like **Adam optimizer** (deep learning).

⚙️ 7. Applications

Field	Use
🧠 Deep Learning	Momentum & Adam optimizers
📈 Finance	Smoothed moving averages
📊 Time Series	Trend detection
🔍 Signal Processing	Noise reduction

8. In summary

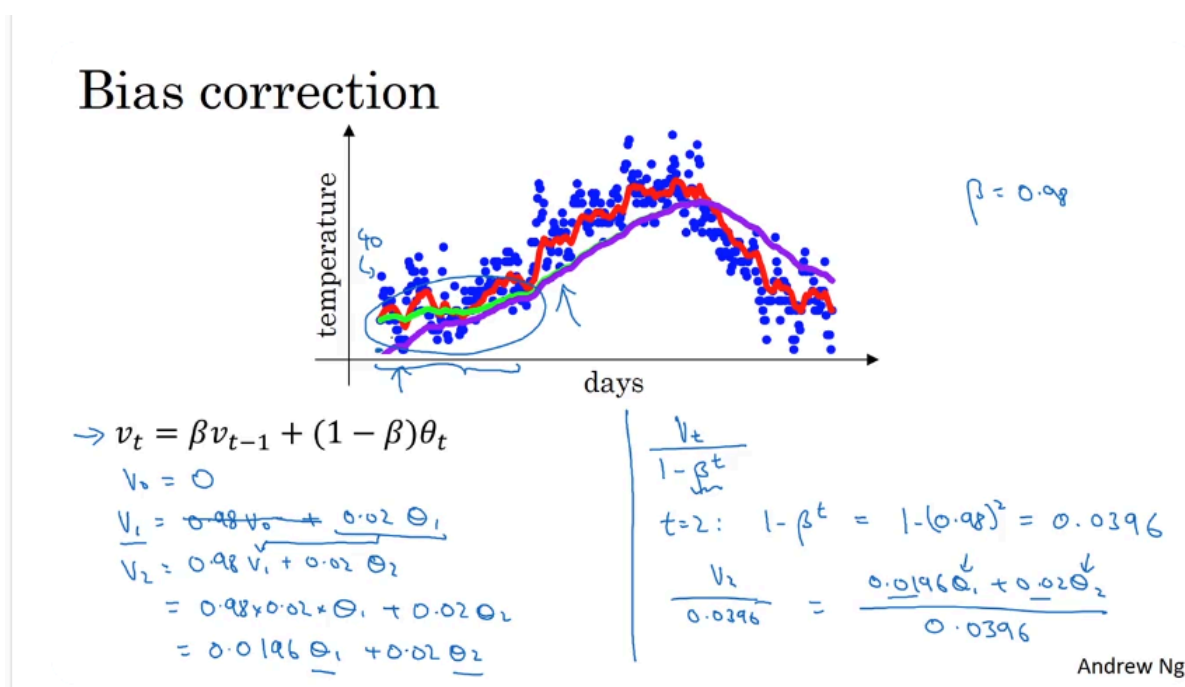
Concept	Description
Idea	Give recent values more weight
Formula	$v_t = \beta v_{t-1} + (1 - \beta)x_t$
Decay factor	β controls smoothness
High β (0.9–0.99)	More smoothing, slower to react
Low β (0.1–0.5)	Less smoothing, reacts quickly

Bias Correction in Exponentially Weighted Averages

when t is a small number like 0, 1, 2, 3

the average value is really small

It turns out that there's a way to modify this estimate that makes it much better, that makes it more accurate, especially during this initial phase of your estimate.



We use the formula:

$$v_t^{\text{corrected}} = \frac{v_t}{1 - \beta^t}$$

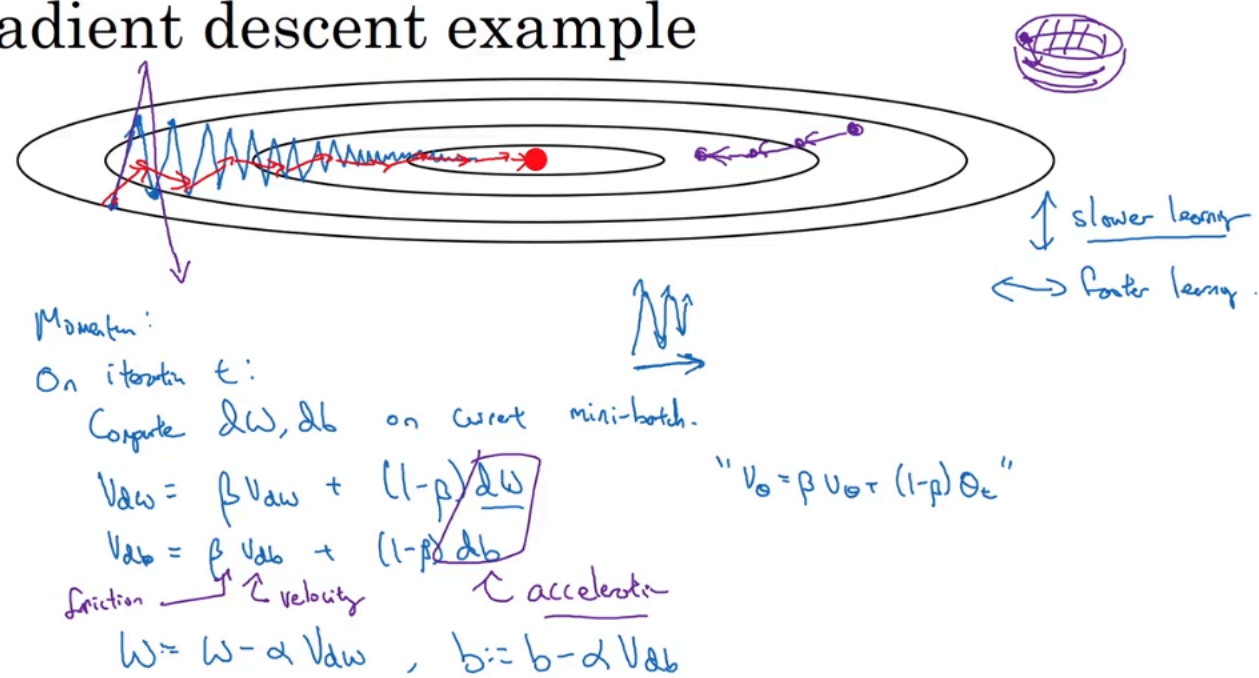
when t is really big, $1 - \beta^t$ is nearly 1, and it make the purple line in the picture goes up to the green line. They overlap each other when 'days' increase

Gradient Descent with Momentum

There's an algorithm called momentum, or gradient descent with momentum that almost always works faster than the standard gradient descent algorithm.

In one sentence, the basic idea is to compute an exponentially weighted average of your gradients, and then use that gradient to update your weights instead.

Gradient descent example



Andrew Ng

- If we choose a really small learning rate, the model will converge really slow (the blue line)
- If we choose a really big learning rate, the model maybe cannot converge (the purple line)
- So that the best line is the red line (go straight to the converge point)

We can think of v is velocity (vận tốc) and gradient of W and b is acceleration (gia tốc) and β is the friction (ma sát)

Implementation details

$$v_{dw} = 0, v_{db} = 0$$

On iteration t :

Compute dW, db on the current mini-batch

$$\begin{aligned} \rightarrow v_{dw} &= \beta v_{dw} + (1-\beta) dW \\ \rightarrow v_{db} &= \beta v_{db} + (1-\beta) db \end{aligned} \quad \left| \quad v_{dw} = \beta v_{dw} + dW \leftarrow$$

$$W = W - \alpha v_{dw}, \quad b = b - \alpha v_{db}$$

Hyperparameters: α, β

$$\beta = 0.9$$

average over last 10 gradients

Andrew Ng

The most common value for Beta is 0.9. We're averaging over the last ten days temperature. So it is averaging of the last ten iteration's gradients. And in practice, Beta equals 0.9 works very well. Feel free to try different values and do some hyperparameter search, but 0.9 appears to be a pretty robust value. Well, and how about bias correction, right? So do you want to take $\frac{v_{dw}}{1-\beta^t}$

In practice, people don't usually do this because after just ten iterations, your moving average will have warmed up and is no longer a bias estimate. So in practice, I don't really see people bothering with bias correction when implementing gradient descent or momentum. And of course, this process initialize the $v_{dw} = 0$.

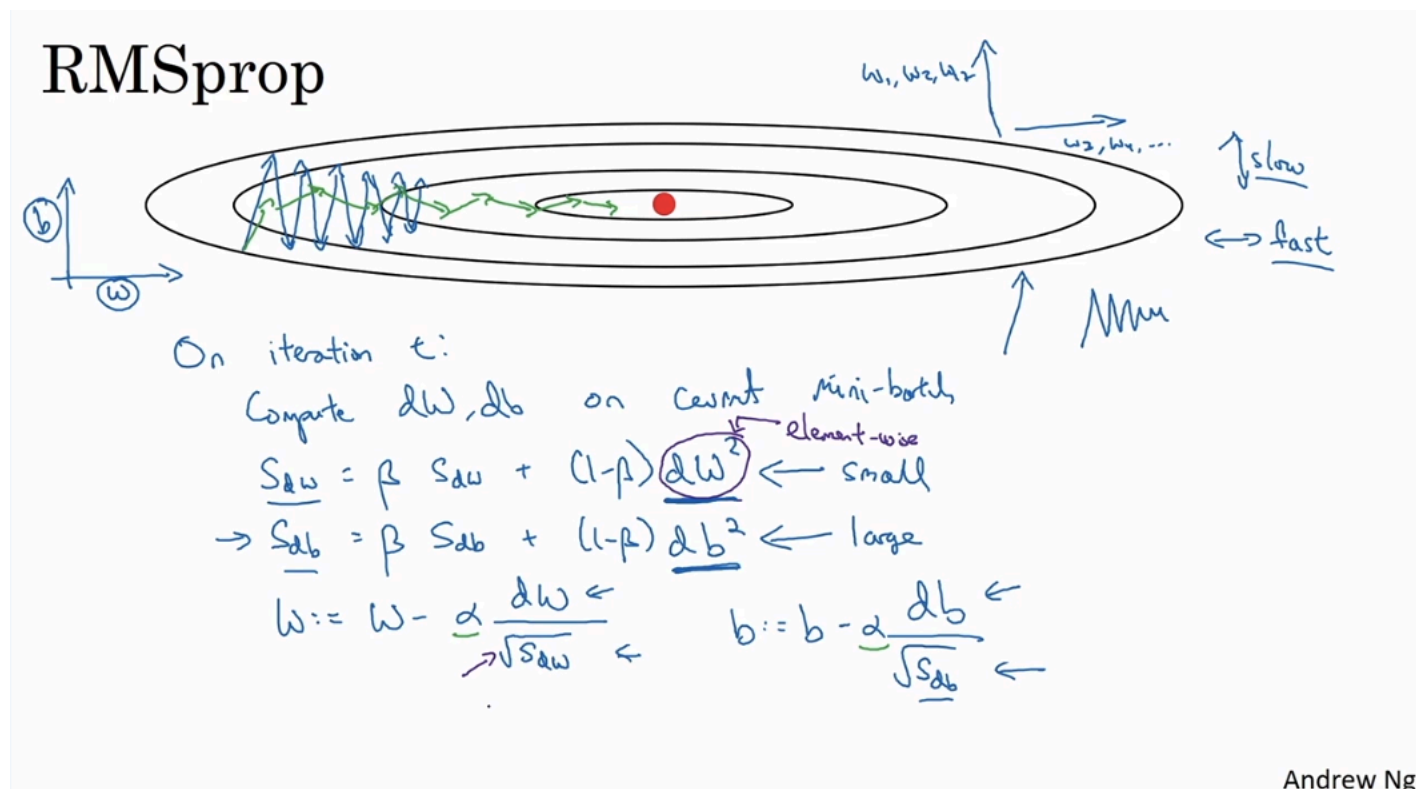
NOTE: This is a matrix of zeroes with the same dimension as dW , which has the same dimension as W . And v_{db} is also initialized to a vector of zeroes. So the same dimension as db , which in turn has same dimension as b .

Finally, I just want to mention that if you read the literature on gradient descent with momentum often you see it with this term omitted, with this $(1 - \beta)$ term omitted.

RMSprop

RMSprop: Root Mean Squared prop

- That can also speed up gradient descent



- The green line represents for the movement of RMSprop
- Where if dW is small, the squared root of S_{dW} also small, which make $\frac{dW}{\sqrt{S_{dW}}}$ is kinda bigger, that make the model come to converge faster
- When db is large → S_{db} is large → $\frac{db}{\sqrt{S_{db}}}$ is smaller, that make model learn slower, and it's more stable

To avoid division by zero:

$$\frac{db}{\sqrt{S_{db} + \epsilon}}$$

- Add a small number epsilon to avoid devididing by zero

$$\epsilon = 10^{-8}$$

Please note that in the next video at 2:44, the following picture is correct. However, later in the video, the db^2 lost the 2 .

The bottom right equation should be:

$$S_{db} = \beta_2 S_{db} + (1 - \beta_2) db^2$$

Adam Optimization Algorithm

→ Combine of Gradient Descent with Momentum and RMSprop

Adam : Adaptive moment estimation

Hyperparameters choice:

→ α : needs to be tune
→ β_1 : 0.9 → ($\underline{dw}$)
→ β_2 : 0.999 → ($\underline{dw^2}$)
→ ϵ : 10^{-8}

Adam: Adaptive moment estimation



Adam Coates

Andrew Ng

Learning Rate Decay

Clarification about Learning Rate Decay Video

Please note that in the next video, the formula for learning rate decay α is:

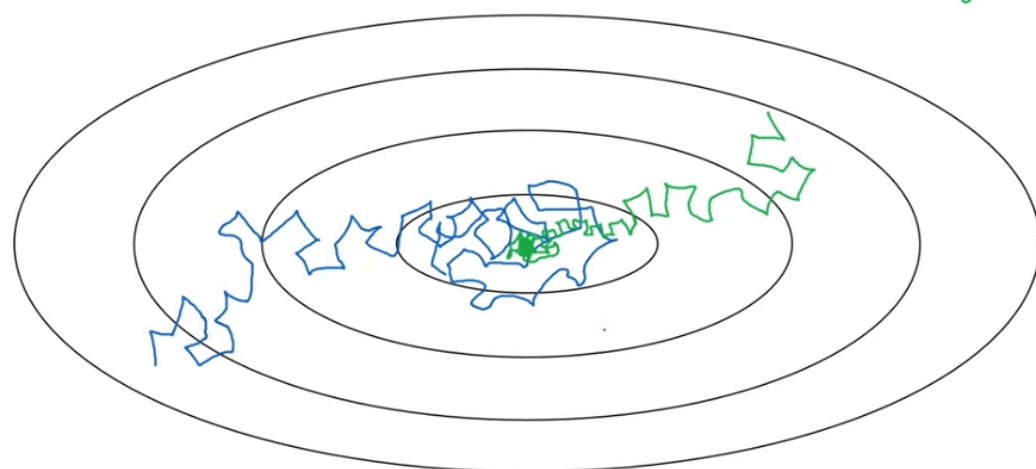
$$\alpha = \frac{1}{1 + \text{decayRate} \times \text{epochNumber}} \alpha_0$$

Mark as completed

If we use fix alpha, maybe it cannot be converge

So that we can: → slowly reduce our learning rate

Learning rate decay



Fixed learning rate → blue line

Slowly reduce learning rate: → green line

Remind: 1 epoch = 1 pass through the data

decay - rate : The hyperparameter that we should tune

Andrew Willing

Epoch	α
1	0.1
2	0.67
3	0.5
4	0.4

But these values will be later **corrected** at 3:31 onwards. Please note these values instead:

Epoch	α
1	0.1
2	0.067
3	0.05
4	0.04

Learning rate decay

1 epoch = 1 pass through data.

$$\alpha = \frac{1}{1 + \text{decay-rate} * \text{epoch-num}} \alpha_0$$

Epoch	α
1	0.1
2	0.067
3	0.05
4	0.04

$\alpha_0 = 0.2$
 $\text{decay-rate} = 1$

$$\alpha = \frac{1}{1 + \text{decayRate} * \text{epochNum}} \alpha_0$$

Where alpha is the learning rate

And:

- Decay Rate is a hyperparameter
- alpha 0 : also a hyperparameter

Other learning rate decay methods

Formula

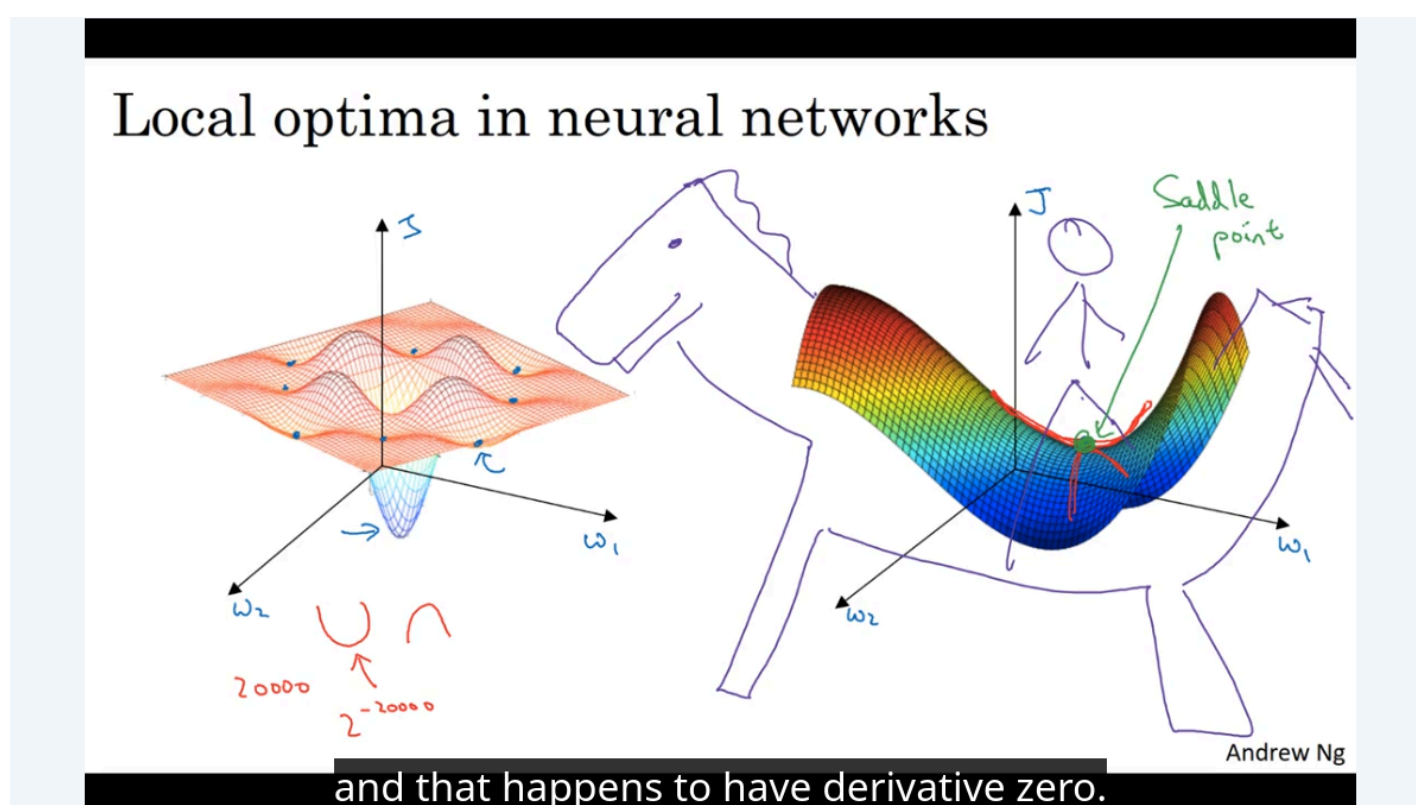
$$\alpha = 0.95^{\text{epoch-num}} \cdot \alpha_0 \quad - \text{exponentially decay.}$$
$$\alpha = \frac{k}{\sqrt{\text{epoch-num}}} \cdot \alpha_0 \quad \text{or} \quad \frac{k}{\sqrt{t}} \cdot \alpha_0$$


discrete staircase

Manual decay.

Andrew Ng

The Problem of Local Optima

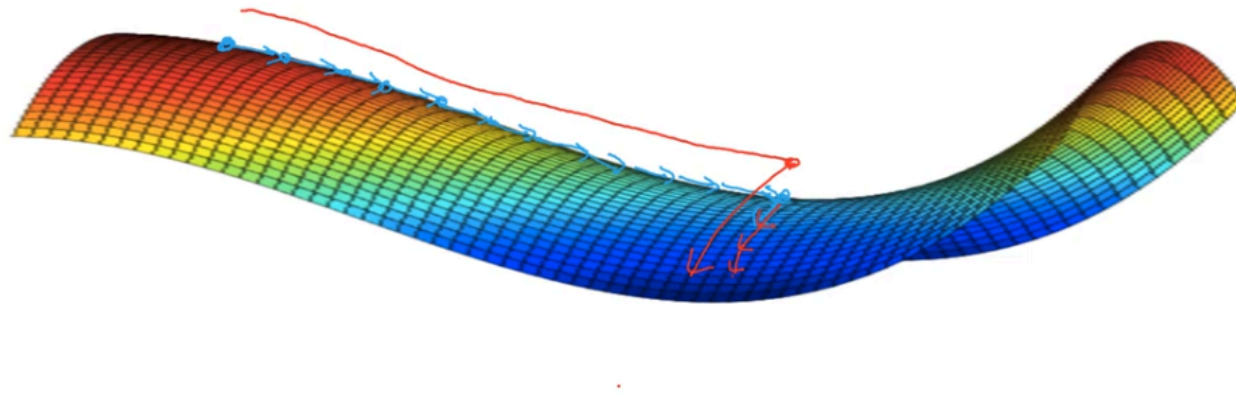


- Saddle point: Where gradient is the zero

But If local optima aren't a problem, then what is a problem?

It turns out that plateaus can really slow down learning and a plateau is a region where the derivative is close to zero for a long time.

Problem of plateaus



- Unlikely to get stuck in a bad local optima
- Plateaus can make learning slow

Andrew Ng

→ this is when Adam can help